정규 스터디

7장
시계열 분석



최민혁

(지도교수: 양선웅)

한양대학교 ERICA 기계공학과

Intelligent Design for Engineering Applications Lab

Part 1 RNN
Part 2 LSTM
Part 3 GRU

RNN(Recurrent Neural Network)

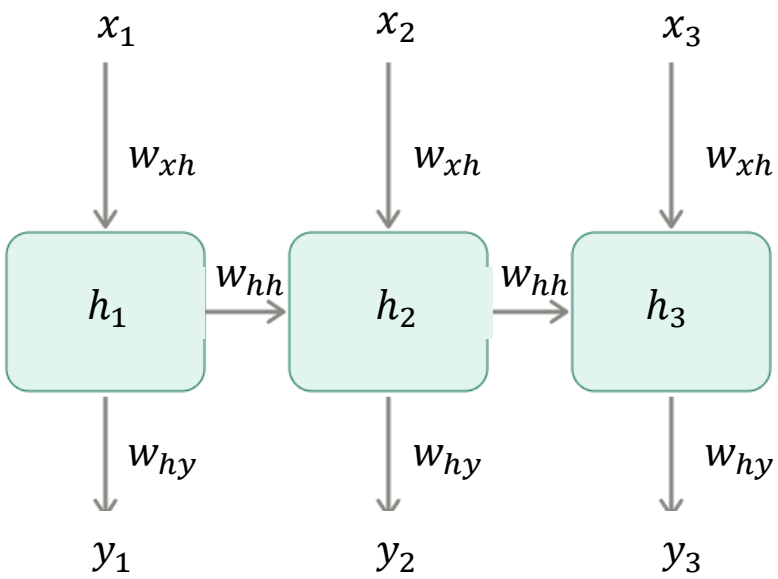
- Memory를 가짐 (한 layer 내부의 $t-1$과 t를 연결하는 weight, bias 존재)
- t 시점 노드를 갱신하기 위해 이전 시점 $t-1$의 hidden state와 t 시점의 input을 활용함

RNN 구조

❖ 각 layer 내부 recurrent의 W, b 는 공유됨

❖ One to one / one to many / many to many

$$h_t = \tanh(y_t)$$
$$y_t = W_{hh} \times h_{t_1} + W_{xh} \times x_t + b$$



RNN 특징

❖ Layer에서 Weight, bias의 공유

- ✓ 변수가 아닌 sequence 상 위치에 대한 독립성
- ✓ 파라미터의 효율성

❖ 역전파 방법

$$\frac{\partial L_t}{\partial W_{hh}} = \sum_{k=1}^{t} \frac{\partial L_t}{\partial h_t} \left(\prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W_{hh}}$$

- ✓ 기울기 소멸 문제
- ✓ Truncated BPTT: 오류 역전파에 이용하는 시점 수를 제한

Part 1 RNN
Part 2 LSTM
Part 3 GRU

RNN(Recuurent Neural Network)

RNN's Hyper Parameter

❖ BPTT Truncation Length
- ✓ Truncated BPTT에서 제한한 시점 길이

❖ Unroll Step
- ✓ 계산 그래프 상에서 RNN이 펼쳐진 횟수

❖ Sequence Length
- ✓ RNN에 한번에 입력되는 time step의 수
- ✓ Unroll step과 맞추기 위해 padding, truncation, masking 등으로 입력 데이터를 처리
- ✓ Masking: Padding 위치의 loss를 0으로 처리

RNN 문제점

❖ Vanishing / Exploding Gradient

$$\frac{\partial h_t}{\partial h_1} = \prod_{k=2}^{t} W_{hh}^T \cdot diag\left(tanh'(\cdot)\right)$$

- ✓ Sequence 초반의 정보가 $W, tanh$의 중첩으로 인해 0에 수렴 또는 발산
- ✓ Truncated BPTT
- ✓ Gradient clipping: 기울기 전체나 원소의 크기를 제한

❖ 병렬화 불가
- ✓ h_t를 계산하기 위해 h_{t-1}이 필요

❖ Information Bottleneck
- ✓ 모든 과거 sequence가 h로 압축되며 전방 정보 손실

Part 1 RNN
Part 2 LSTM
Part 3 GRU

LSTM (Long Short-Term Memory)

- 일반 RNN의 Gradient Vanishing을 해결하기 위해 고안됨

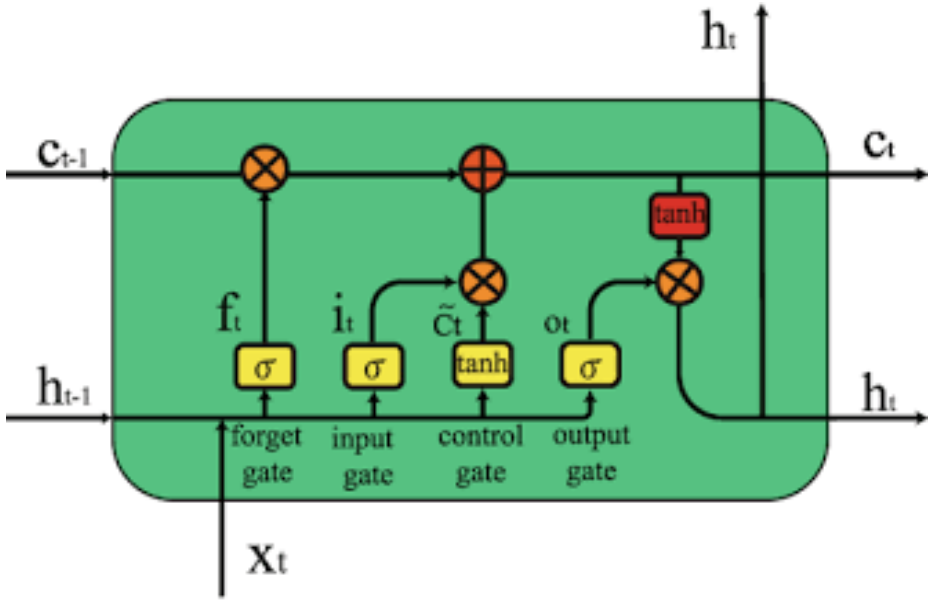
LSTM 구조

❖ RNN에 과거 정보를 보존하는 cell state 도입함

❖ Cell state (Long-Term Dependency)

✓ x_t가 덧셈으로 합류되어 vanishing gradient를 회피

✓ 과거 정보 반영 여부를 forget gate가 판단



LSTM 특징

❖ Forget gate: 과거 정보 보존량 결정

$$c_t = f_t \cdot c_{t-1}$$
$$= \sigma(w_f[h_{t-1}, x_t] + b_f) \cdot c_{t-1}$$

❖ Input gate: 현재 정보 보존량 결정

$$c_t = c_{t-1} + i_t \cdot \tilde{c}_t$$
$$= c_{t-1} + \sigma(w_i[h_{t-1}, x_t] + b_i) \cdot \tanh(w_c[h_{t-1}, x_t] + b_C)$$

$\tilde{c}_t$: Candidate value

❖ Output gate: 출력 결정

$$h_t = o_t \cdot \tanh(c_t)$$
$$= \sigma(w_o[h_{t-1}, x_t] + b_o) \cdot \tanh(c_t)$$

$0 < \sigma < 1$: 반영 또는 유지할 비율

$-1 < \tanh < 1$: 반영할 정보. 값을 압축하면서 음수 및 비선형성을 표현할 수 있는 방법

[1] J. Luo, Z. Zhang, Y. Fu, and F. Rao, "Time series prediction of COVID-19 transmission in America using LSTM and XGBoost algorithms," Jun. 2021. [Online]. Available: Time series prediction of COVID-19 transmission in America using LSTM and XGBoost algorithms

LSTM (Long Short-Term Memory)

LSTM's Learnable Parameter

❖ W_f
- ✓ W_f: x_t에 대한 weight
- ✓ U_f: h_{t-1}에 대한 weight

❖ b_f
- ✓ Bias

❖ m: Hidden dimension

❖ n: Input dimension

Patameter	W	U	b
Forget gate f_t	$m \times n$	$m \times m$	m
Input gate i_t	$m \times n$	$m \times m$	m
Candidate value $\tilde{c}_t$	$m \times n$	$m \times m$	m
Output gate o_t	$m \times n$	$m \times m$	m

❖ LSTM Layer 하나가 가지는 Learnable parameter

$$4m(m+n+1)$$

LSTM 학습의 특징

❖ 모든 시점에서 weight, bias 공유

❖ 순차적 계산으로 인한 병렬화 불가

❖ Exploding Gradient
- ✓ Gradient clipping

❖ 여러 gate의 vanishing gradient
- ✓ Forget: 장기기억 무력화 / 망각 불가
- ✓ Input: 학습 불가
- ✓ Output: 표면적 학습

❖ Uninterrupted gradient flow

$$\frac{\partial C_t}{\partial C_t} = f_t$$

- ✓ RNN 대비 long term에 대한 학습

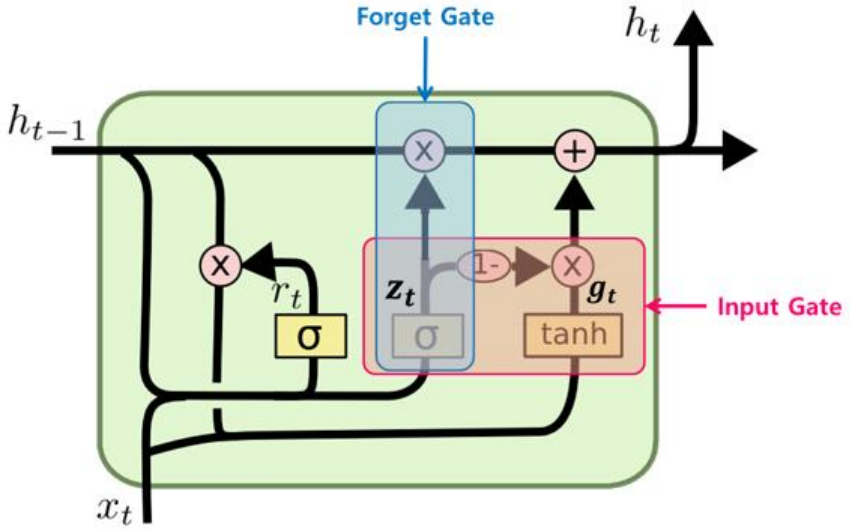
Part 1 RNN
Part 2 LSTM
Part 3 GRU

GRU (Gated Recurrent Unit)

- Gate controller가 forget gate와 input gate를 모두 제어
- Cell State와 Hidden State를 통합함

GRU 구조

❖ LSTM과 유사하나, forget gate와 input gate가 합쳐져 update gate로 통합



GRU 특징

❖ Reset gate(r_t):

✓ Candidate를 만들 때 과거를 얼마나 참조할지 결정

❖ Update gate:

✓ 현 시점의 정보량을 z, 직전 시점 정보를 $1 - z$로 반영

$$r_t = \sigma\left(W^{(r)}x_t + U^{(r)}h_{t-1}\right)$$

$$z_t = \sigma\left(W^{(z)}x_t + U^{(z)}h_{t-1}\right)$$

$$\tilde{h}_t = \tanh\left(Wx_t + r_t \circ Uh_{t-1}\right)$$

$$h_t = z_t \circ h_{t-1} + (1 - z_t) \circ \tilde{h}_t$$

[2] Hyen4110, "[딥러닝] GRU(Gated Recurrent Unit)," *Hyen4110*, May 10, 2021. [Online]. Available: https://hyen4110.tistory.com/26. [Accessed: Jul. 21, 2026].

Part 1
RNN

Part 2
LSTM

Part 3
GRU

GRU (Gated Recurrent Unit)

GRU 특징

❖ Learnable parameter

$3m(m+n+1) < 4m(m+n+1)$ (LSTM's learnable parameter)

RNN:LSTM:GRU=1:4:3

❖ 이론적으로 LSTM보다 상대적으로 빠르나 표현력이 다소 떨어짐

- ✓ Patameter의 수에 따른 속도와 표현력
- ✓ 성능 우열은 학습조건에 따라 달라질 수 있음

시계열 Deep Learning 모델 비교 및 요약

Intelligent Design for Engineering Applications Lab

RNN

구조

- h_t 하나만 순환하는 단일 tanh layer

Gate

- 없음

Parameter

- 가장 적음→ 계산 비용 최소

한계

- Sequence가 길어지면 vanishing / exploding gradient 발생
- Long-term dependency 학습 어려움

적합한 경우

- 계산 비용이 극히 제한된 경우

LSTM

구조

- h_t 와 C_t 를 분리해 유지

Gate

- Forget gate, input gate, output gate

Parameter

- RNN의 약 4배 → 계산 비용 최대

특징

- Cell state가 장기기억을 전달해 vanishing gradient 완화
- Long-term dependency 보존에 가장 강함

적합한 경우

- 긴 sequence, 데이터가 충분한 대규모 학습

GRU

구조

- C_t 없이 h_t로 통합

Gate

- Reset gate, update gate

Parameter

- RNN의 약 3배 → LSTM 대비 빠른 학습

특징

- Forget gate, input gate를 합쳐 비교적 단순한 구조로 갱신량을 조절
- 성능은 LSTM과 대등한 수준

적합한 경우

- 데이터가 적거나 빠른 학습이 필요한 경우

LSTM vs GRU : 표현력 차이의 근원

Intelligent Design for Engineering Applications Lab

LSTM vs GRU

1. 기억과 출력의 분리

- LSTM : cell state C_t(기억) 와 hidden state h_t(출력) 를 분리, output gate 로 노출량을 따로 조절

$$h_t = o_t \odot \tanh(C_t)$$

- GRU : 상태가 하나 → 기억 = 출력. 정보를 보관하려면 계속 노출해야 하고, 감추려면 잃어야 함
- 효과 : 먼 시점에서만 쓸 정보를 중간 구간에 감춰 둘 수 있는지의 차이 (간섭 억제)

2. 갱신 방식 : 덧셈 누적 vs 볼록결합

$LSTM\ C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$

$GRU\ \ h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$

- LSTM 은 forget · input 이 독립 → 보존과 추가를 동시에 수행, 상태값이 유계가 아님
- GRU 는 게이트 하나가 둘을 겸함 → z 만큼 쓰려면 z 만큼 버려야 하고, 상태는 항상 유계

3. 표현력 상한의 차이

- GRU 는 상태가 유계이므로 임의 길이의 counting 을 표현할 수 없음 → 중첩 괄호, 파싱, 깊이 추적 계열 과제에서 격차
- layer 를 쌓아 파라미터를 맞춰도 이 제약은 사라지지 않음 (용량 문제가 아니라 구조 문제)
- 긴 보존 + 활발한 갱신을 동시에 요구하는 구간에서 LSTM 이 유리

4. 실무적 함의

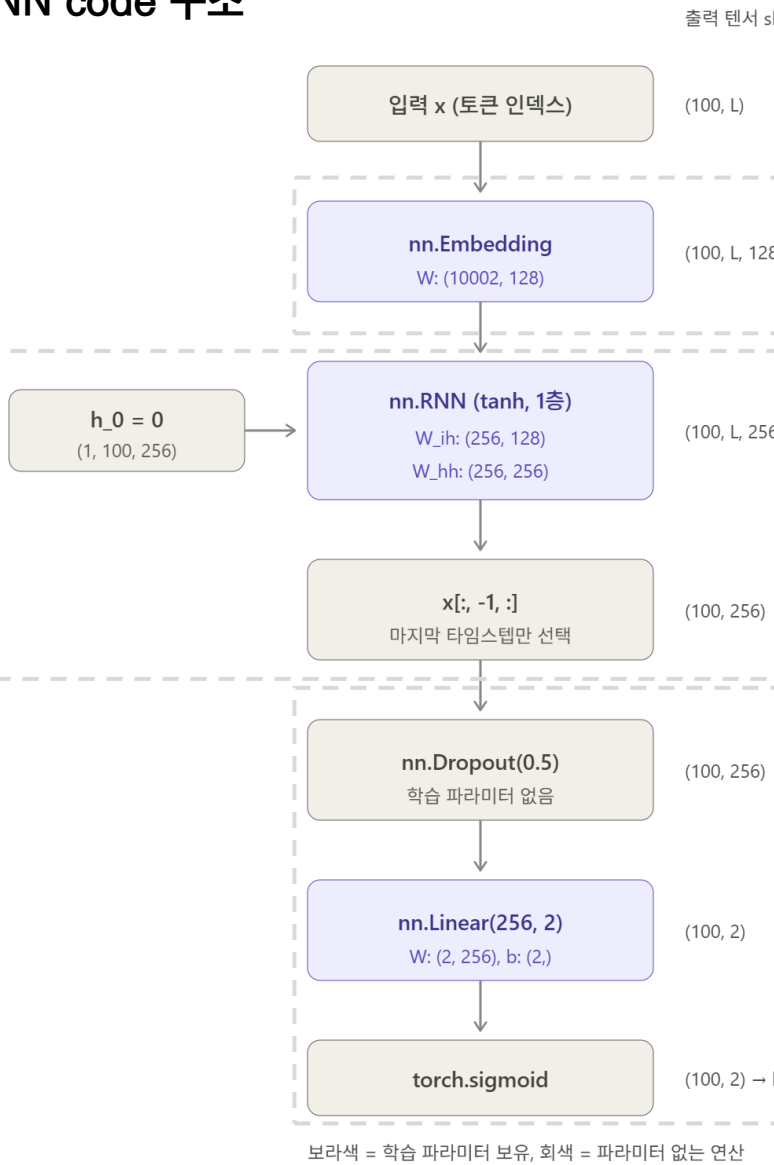
- 차이가 드러나는 조건 : 긴 sequence, 계층적 의존성, 충분한 데이터, 학습보다 긴 길이로의 일반화
- 차이가 사라지는 조건 : 짧은 sequence, 적은 데이터 → 동일 파라미터 예산이면 GRU 가 대등하거나 우세
- 실제 벤치마크 격차는 hidden size · dropout · learning rate 튜닝 차이에 묻힐 정도로 작음
- 정리 : “LSTM 이 강력하다” 는 표현력 상한에 대한 진술이며, 평균 성능에 대한 진술은 아님

요약

과적합 위험은 파라미터 예산을 맞추면 비슷해지지만, 상태 유계성에서 오는 표현력 상한 차이는 layer 를 쌓아도 남는다 — 선택 기준은 sequence 길이와 데이터 규모다

시계열 Deep Learning 모델 Code

교제 RNN code 구조



100개의 영화 소감문을 보고 긍정 평가인지 부정 평가인지 평가한 (100, 2) 행렬 출력
순환 과정에서 parameter를 공유해 소감문 길이가 무관

각 토큰에 대응하는 벡터로 변환

가장 긴 문장의 길이만큼 순환 (길이가 부족한 경우 padding)

$$h_t = \tanh(W_{ih} \cdot x_t + b_{ih} + W_{hh} \cdot h_{t-1} + b_{hh})$$

Batch: 100개, $(256, 128) \cdot (128, 1)$

$\Rightarrow (100, 256)$

Batch: 100개, $(256, 256) \cdot (256, 1)$

$\Rightarrow (100, 256)$: Hyper parameter

시계열 모델 선택에 따라 달라지는 부분

0.5로 dropout 적용 후

$(100,256) \cdot (256, 2) + bias$

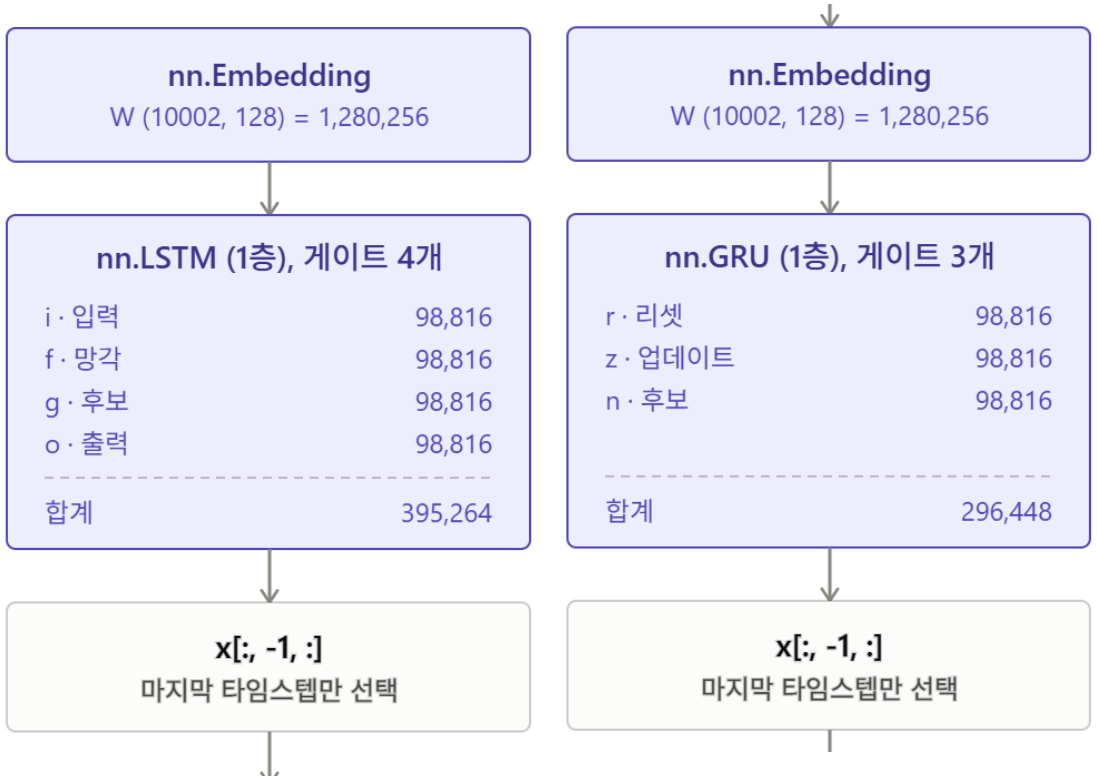
$\Rightarrow (100, 2)$

100개 데이터의 (긍정, 부정) 점수 도출 후
큰 쪽이 1, 작은 쪽이 0

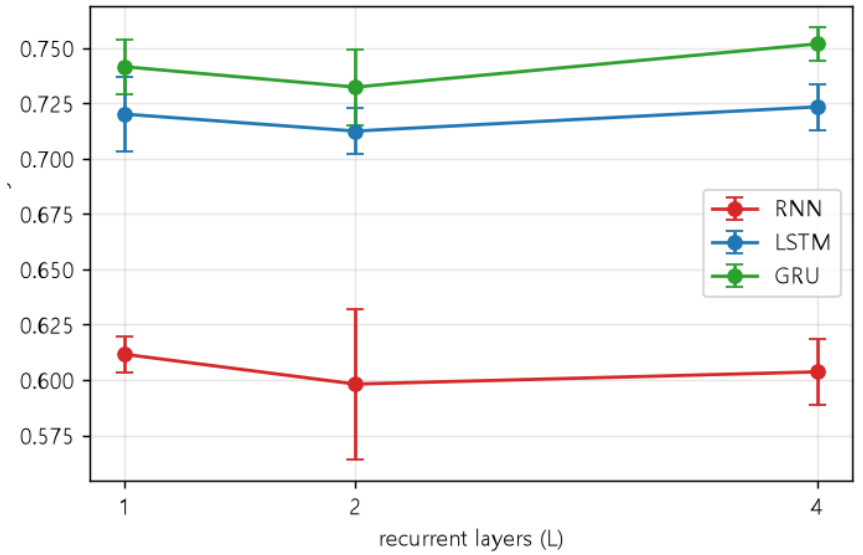
$$loss = -log\left(\frac{e^{logit[y]}}{e^{logit[0]} + e^{logit[1]}}\right) \quad (Cross\ Entropy)$$

시계열 Deep Learning 모델 Code

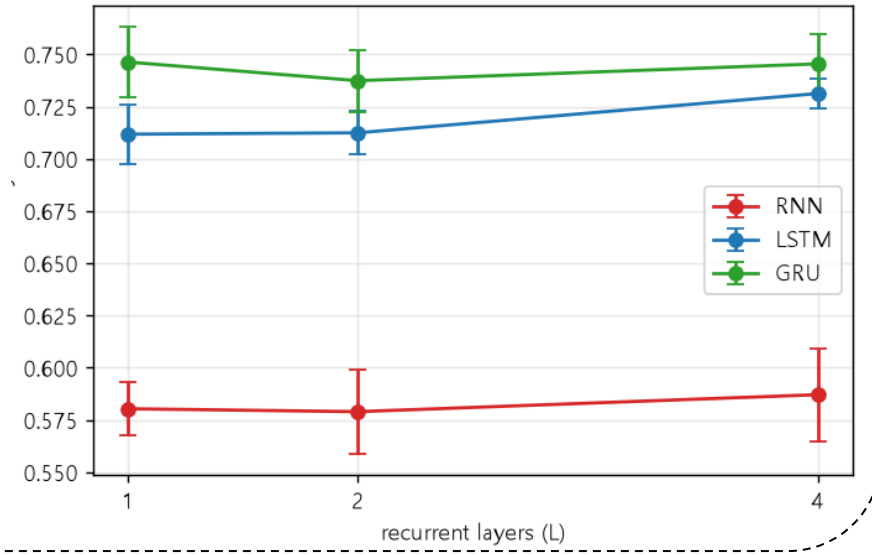
교제 RNN code 구조 _ LSTM, GRU로 변형



Layer 차원 크기
유지하고 Layer 쌓기



Parameter 수
유지하고 Layer 쌓기



시계열 Deep Learning 모델 Code

Code

Q. 입출력 구조가 '~ to many'일 때 가변 길이의 nn.Linear에서 parameter의 차원이 불일치?
A. Output에 대해 단순 flatten을 진행하지 않음.
(각 recurrent의 output 평균을 이용하거나 output끼리 weight, bias 행렬을 공유)
(위치가 중요할 때 flatten이 합리적이며, 이 때는 sequence 길이 고정)

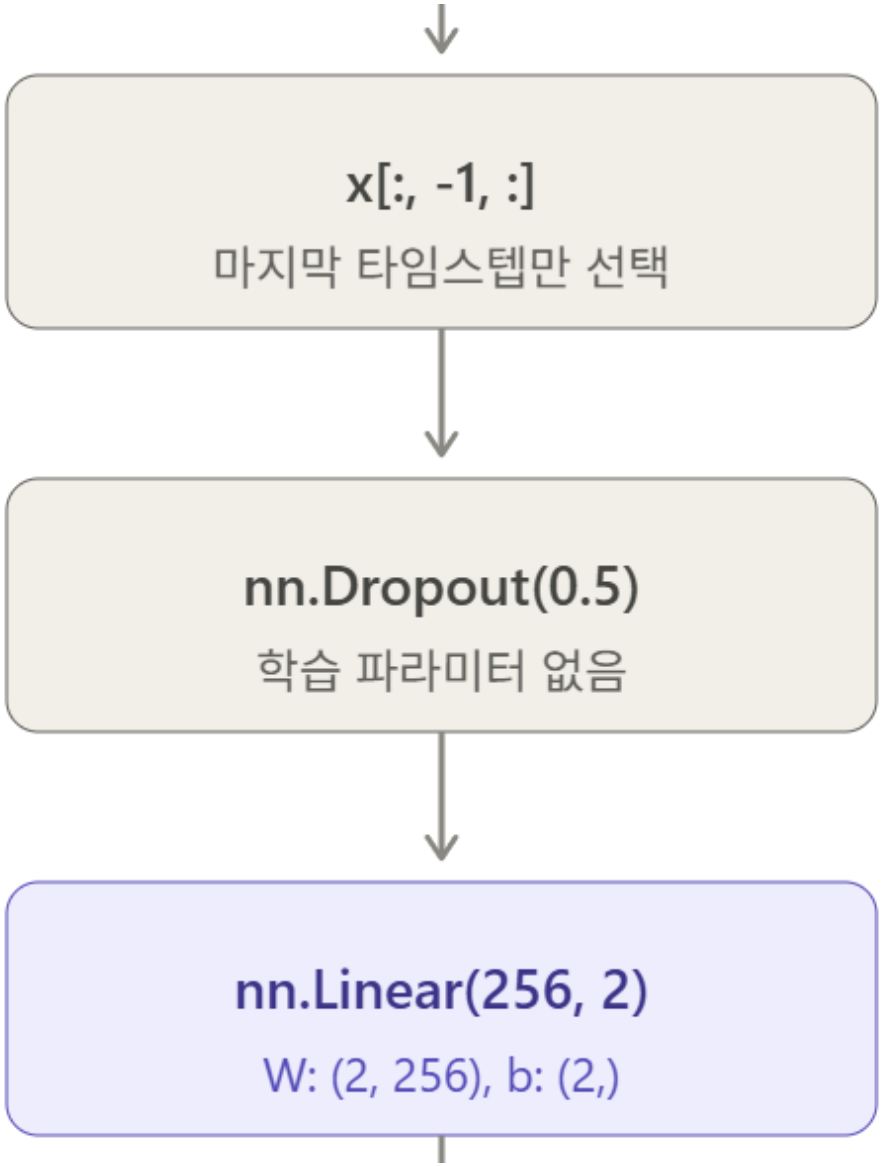
Q. RNN layer output이 모델 output과 불일치
A. Output 형식 이외에도 목적에 따른 입출력 구조 선택

시계열 데이터의 전처리

1. **시간 간격 입력**
   - ✓ 불균일 시계열 데이터

2. **결측치 데이터 표기법**
   - ✓ 결측치와 결측치를 대체한 값을 구분

3. **Padding**
   - ✓ 시계열 데이터의 길이 통일



세 시계열 Deep Learning 모델의 한계와 Attention

premature forgetting _조기 망각

❖ 무엇이 중요한지가 미래에서 결정

e.g. 시점 $t-1$에서 $t-2$의 정보가 필요 없다고 판단하면 t에서 $t-2$의 정보가 필요해도 복구할 수 없다.

❖ RNN 계열:

✓ 순전파 시점의 표현력 문제

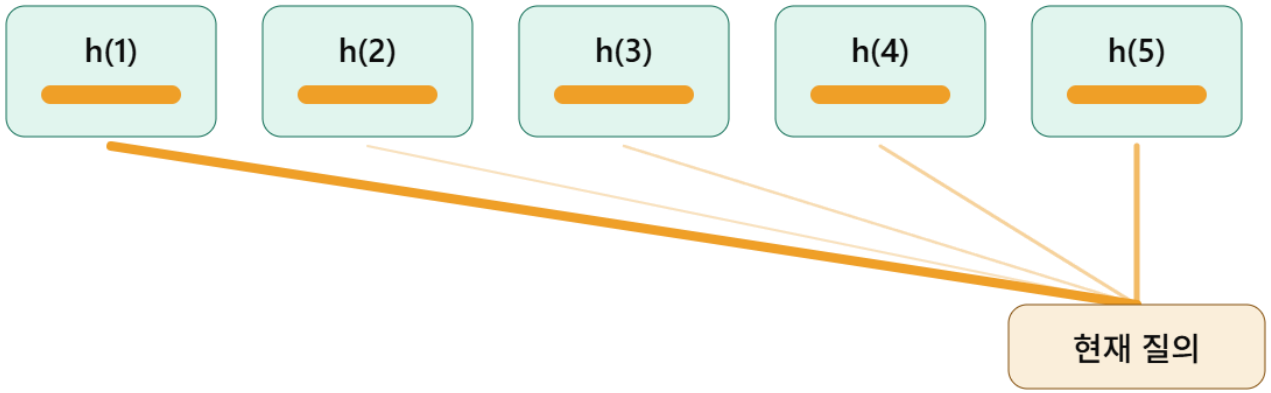
❖ Attention:

✓ 각 토큰이 다른 토큰에 대해 “의미를 정하는 데 누가 얼마나 중요한가”를 가중치로 매기고, 그 가중치대로 정보를 혼합

RNN 계열

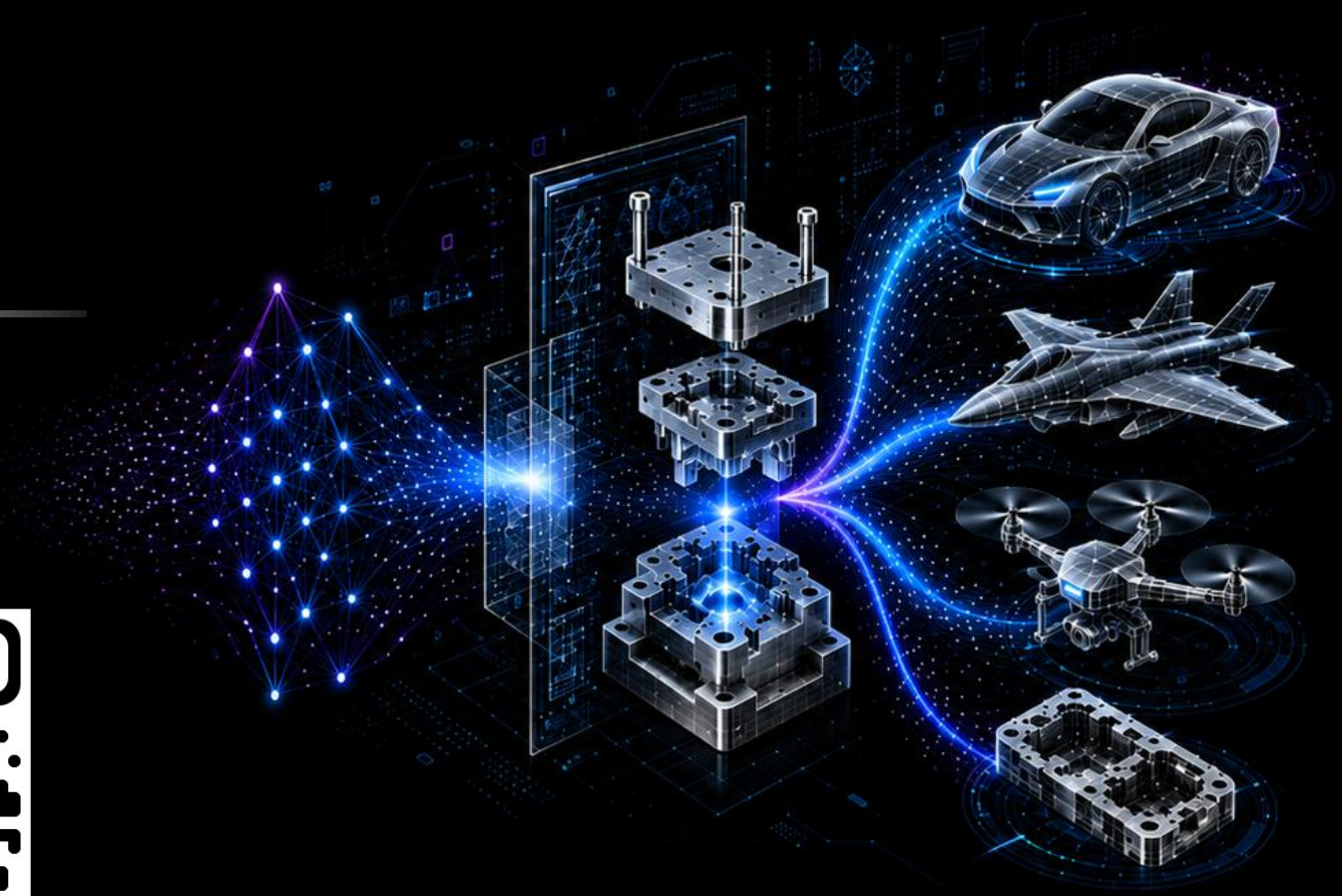


Attention



주황 막대 = t=1 시점 정보의 잔존량

Any Questions?



이름 최 민 혁

소속 한양대학교 기계공학과

이메일 mindat@hanyang.ac.kr

Intelligent Design for Engineering Applications Lab